

Aprendizaje de Máquinas

Introducción, KNN y Naive Bayes

Jorge Gallego

Pontificia Universidad Javeriana

Bogotá Summer School in Economics — 2026

Agenda de la sesión

1. Introducción al aprendizaje de máquinas

- ▶ ¿Qué es? ¿Para qué sirve?
- ▶ Causalidad vs. predicción
- ▶ Tipos de aprendizaje y aplicaciones

2. Clasificación con vecinos más cercanos (KNN)

3. Aprendizaje probabilístico: Naive Bayes

- En cada técnica: intuición en las diapositivas y un taller práctico en R

Parte 1

Introducción al aprendizaje de máquinas

Machine learning en la vida cotidiana

- Usamos aprendizaje de máquinas todos los días, casi sin notarlo
- El filtro de spam de Gmail
- Los asistentes de voz como Alexa o Siri
- Las recomendaciones de Netflix o Spotify
- Los carros sin conductor
- La evaluación de riesgo crediticio o la detección de fraude

Filtros de spam



Carros sin conductor



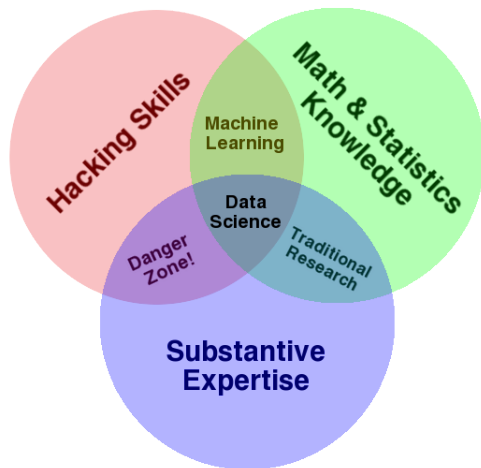
¿Qué tienen en común?

- Todos se basan en grandes volúmenes de información (*big data*)
- Que proviene de diversas fuentes y se almacena sistemáticamente
- Y que se procesa con técnicas matemáticas, estadísticas y computacionales
- A ese conjunto de técnicas lo llamamos *Machine Learning*

¿Qué es Machine Learning?

- Es un campo de la inteligencia artificial
- Estudia algoritmos que **transforman los datos en acción inteligente**
- Son sistemas que **mejoran su desempeño con la experiencia**
- No es que por primera vez tengamos muchos datos: siempre los hemos tenido
- Lo nuevo es nuestra capacidad de almacenarlos y de analizarlos mejor

Un campo en la intersección



La ciencia de datos combina estadística, computación y conocimiento del dominio

Aprendizaje supervisado y no supervisado

- **Aprendizaje supervisado:** buscamos predecir una característica de interés (el *outcome*) en función de las demás
 - ▶ Partimos de ejemplos ya “etiquetados” para entrenar
 - ▶ Ejemplo: predecir si un correo es spam usando su contenido
- **Aprendizaje no supervisado:** buscamos describir o resumir los datos
 - ▶ Ninguna característica es más importante que las demás
 - ▶ Ejemplo: agrupar clientes con comportamiento similar

Tipos de problemas

- **Clasificación** (supervisado): predecir una categoría
 - ▶ ¿Es spam o no? ¿La célula es benigna o maligna? ¿Hará default?
- **Predicción numérica** (supervisado): predecir un valor continuo
 - ▶ Ingreso, precio de una vivienda, gasto en salud
- **Clustering** (no supervisado): dividir los datos en grupos homogéneos
- **Descubrimiento de patrones** (no supervisado): asociaciones, anomalías

Causalidad vs. Predicción

“Los datos son el nuevo petróleo; como el petróleo, deben refinarse antes de usarse” (Andreas Weigend)

- Los datos son cruciales para el diseño y la evaluación de políticas públicas
- Pero, ¿cómo usarlos según el caso?
- No es lo mismo querer **explicar** que querer **pronosticar**
- Veamos dos ejemplos de “juguete”

Causalidad vs. Predicción

¿Podemos hacer llover si invertimos en la “danza de la lluvia”?



Causalidad vs. Predicción

¿Es beneficioso salir hoy a la calle con sombrilla?



Causalidad vs. Predicción

- Son dos preguntas distintas sobre el clima
- En la primera, importa la **causalidad**: ¿la danza *causa* la lluvia?
- En la segunda, importa la **predicción**: ¿es tan probable que llueva como para sacar la sombrilla?
- En argot de política pública: **Evaluación de Impacto** vs. **Big Data**
- Este curso se concentra en el segundo problema: **predecir bien**

Machine learning para políticas públicas

Algunas preguntas donde la predicción ayuda a decidir:

- **Justicia:** ¿qué tan probable es que un sospechoso reincida?
- **Seguridad:** ¿dónde ocurrirán los próximos delitos (*hotspots*)?
- **Salud:** ¿a qué pacientes conviene asignar un tratamiento costoso?
- **Corrupción:** ¿qué contratos públicos tienen mayor riesgo de terminar mal?

Seguridad: predicción de crimen



Daniel E. Ortega

@dortegaeval

Siguiendo



#PuntosCalientesBogotá "Ni en Europa ni en EE. UU. ha habido una intervención en puntos calientes de esa magnitud" goo.gl/9R9Y3j



RETWEET

1

ME GUSTA

4



Ética y Machine Learning

- Estas técnicas no están libres de riesgos
- Los **falsos positivos** pueden ser muy costosos (un sospechoso, un paciente)
- Los algoritmos pueden **heredar sesgos** de los datos con que se entrenan
- Y está el tema de la **privacidad**: ¿cuánto estamos dispuestos a ceder?
- El humano sigue siendo necesario para motivar el análisis y dar sentido a los resultados

El mapa del curso

- I. Aprendizaje supervisado: **clasificación** (KNN, Naive Bayes, SVM, árboles)
 - II. Aprendizaje supervisado: **predicción numérica** (regresión, Ridge, Lasso)
 - III. **Desempeño** de modelos (evaluación, variables, desbalance, ensamblaje)
 - IV. Aprendizaje **no supervisado** (clustering, anomalías, representación)
 - V. Aprendizaje **profundo** (redes neuronales: ANNs, CNNs, RNNs)
- Hoy arrancamos con la Parte I

Herramientas

- Trabajaremos con el lenguaje **R** y el entorno **RStudio**
- La parte conceptual va en estas diapositivas; la práctica, en *notebooks* (`.Rmd`)
- La filosofía: usar paquetes que hacen el trabajo pesado, pero entendiendo **paso a paso** qué hace cada línea
- Cada técnica tiene su propio taller con datos reales

Parte 2

Clasificación con vecinos más cercanos (KNN)

Un principio básico

- Para clasificar partimos de una idea muy intuitiva
- **Las cosas que son parecidas tienen propiedades parecidas**
- “Blanco es, gallina lo pone, frito se come”
- Los algoritmos de *machine learning* usan este principio para clasificar
- En una misma categoría caen los elementos similares, o *vecinos más cercanos*

El algoritmo kNN

- El más popular es **kNN** (*k nearest neighbors*, k vecinos más cercanos)
- Ventajas:
 - ▶ Simple y efectivo
 - ▶ No hace supuestos sobre la distribución de los datos
 - ▶ Fase de entrenamiento muy rápida
- Desventajas:
 - ▶ No produce un modelo: es difícil ver la relación entre variables
 - ▶ Requiere elegir bien el valor de k
 - ▶ Fase de clasificación lenta

¿Cómo clasifica kNN?

- Partimos de un conjunto de datos ya clasificados (entrenamiento)
- Elegimos un número k de vecinos
- Para cada caso **sin clasificar**, buscamos sus k vecinos más cercanos
- El caso se asigna a la categoría de la **mayoría** de esos vecinos

Ejemplo: clasificar alimentos

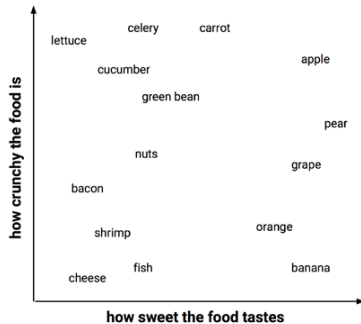
- A una persona con los ojos vendados se le pide clasificar un alimento
- Tenemos una base de otros alimentos, descritos por dos características:
 - ▶ Qué tan crujiente es (de 1 a 10)
 - ▶ Qué tan dulce es (de 1 a 10)
- Y sabemos su grupo: fruta, vegetal o proteína

Ejemplo: clasificar alimentos

Ingredient	Sweetness	Crunchiness	Food type
apple	10	9	fruit
bacon	1	4	protein
banana	10	1	fruit
carrot	7	10	vegetable
celery	3	10	vegetable
cheese	1	1	protein

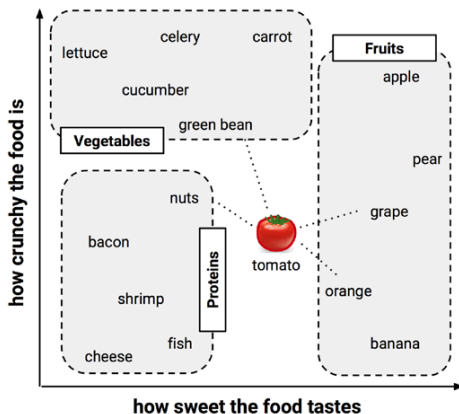
Ejemplo: clasificar alimentos

¿Se reconoce algún patrón?



Ejemplo: clasificar alimentos

¿El tomate es fruta o verdura?



Distancia euclídea

- Para hallar los vecinos más cercanos necesitamos una medida de distancia
- La más usada es la **distancia euclídea**: la longitud del segmento que une dos puntos
- Entre dos vectores m -dimensionales, p y q :

$$\text{dist}(p, q) = \sqrt{\sum_{i=1}^m (p_i - q_i)^2}$$

Distancia al tomate

- Calculamos la distancia del tomate a cada alimento:

Ingredient	Sweetness	Crunchiness	Food type	Distance to the tomato
grape	8	5	fruit	$\text{sqrt}((6 - 8)^2 + (4 - 5)^2) = 2.2$
green bean	3	7	vegetable	$\text{sqrt}((6 - 3)^2 + (4 - 7)^2) = 4.2$
nuts	3	6	protein	$\text{sqrt}((6 - 3)^2 + (4 - 6)^2) = 3.6$
orange	7	3	fruit	$\text{sqrt}((6 - 7)^2 + (4 - 3)^2) = 1.4$

- Con $k = 1$, el vecino más cercano es la naranja: sería **fruta**
- Con $k = 3$ (naranja, uva, nueces): también **fruta**

La elección de k

- Cuántos vecinos usar es una decisión crucial: hay un *trade-off* sesgo–varianza
- k **grande**: menos sensible al ruido, pero puede ignorar patrones pequeños
- k **pequeño**: capta detalles, pero una observación ruidosa puede dañar la clasificación
- No hay regla universal. Una heurística común es $k = \sqrt{n}$
- Mejor aún: probar varios k y quedarse con el que clasifique mejor

Preparación de los datos

- Para calcular distancias, todas las características deben estar en la **misma escala**
- Si no, una variable con valores grandes dominaría solo por su escala
- Dos formas de rescalar:

Normalización min-max:
$$X' = \frac{X - X_{min}}{X_{max} - X_{min}}$$

Estandarización (z-score):
$$X' = \frac{X - \mu}{\sigma}$$

Preparación de los datos

- La **normalización min-max** deja todos los valores entre 0 y 1
- La **estandarización** mide cuántas desviaciones estándar se aleja un valor de la media
- ¿Y las variables nominales? La distancia euclídea no está definida para ellas
- Solución: crear variables *dummy* (si hay n categorías, se crean $n - 1$ dummies)

kNN: aprendizaje perezoso

- A kNN se le llama un algoritmo **perezoso** (*lazy*)
- No hay abstracción: no se construye un modelo, solo se almacenan los datos
- Es una técnica **no paramétrica**: no supone una forma funcional
- Ventaja: flexible. Desventaja: no entendemos cómo se relacionan las variables con el *outcome*
- **AI taller**: diagnóstico de cáncer de seno con la base `wisc_bc_data.csv`

Parte 3

Aprendizaje probabilístico: Naive Bayes

Pronósticos probabilísticos

- A veces la economía se parece más a la meteorología que a la física
- Un meteorólogo no dice “va a llover”; dice “hay 70% de probabilidad de lluvia”
- Basa su pronóstico en eventos pasados
- El método de hoy hace justamente eso: pronostica de manera **probabilística**
- Usa datos del pasado para anticipar eventos cuyo estado desconocemos

Naive Bayes

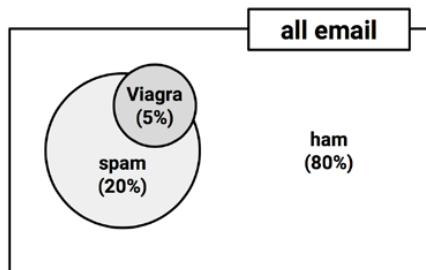
- El algoritmo **Naive Bayes** se basa en el **teorema de Bayes**
- Usa datos de entrenamiento para calcular la probabilidad de un resultado
- Esa probabilidad depende de las características del objeto a clasificar
- Se aplica mucho en:
 - ▶ Clasificación de textos (filtros de spam)
 - ▶ Detección de anomalías o intrusiones en redes
 - ▶ Diagnóstico médico a partir de síntomas

Un poco de probabilidad

- Sea $P(A)$ la probabilidad de que ocurra el evento A
- Su complemento $\neg A$ es la no ocurrencia de A , con $P(\neg A) = 1 - P(A)$
- Dos eventos son **independientes** si la ocurrencia de uno no informa sobre el otro: $P(A \cap B) = P(A) \cdot P(B)$
- Pero los casos interesantes son los **dependientes**: ahí un evento sí nos dice algo del otro

Spam y la palabra “Viagra”

¿Qué nos dice sobre la probabilidad de spam el que un correo contenga “Viagra”?



El teorema de Bayes

- La probabilidad condicional de A dado B es:

$$P(A | B) = \frac{P(A \cap B)}{P(B)}$$

- Reorganizando ($P(A \cap B) = P(B | A) \cdot P(A)$), llegamos a la forma más útil:

$$P(A | B) = \frac{P(B | A) \cdot P(A)}{P(B)}$$

- Nos permite “actualizar nuestra creencia” sobre A al observar B

Ejemplo: spam y Viagra

$$P(\text{spam} \mid \text{Viagra}) = \frac{P(\text{Viagra} \mid \text{spam}) \cdot P(\text{spam})}{P(\text{Viagra})}$$

- $P(\text{spam} \mid \text{Viagra})$: probabilidad *posterior*
- $P(\text{spam})$: probabilidad *a priori*
- $P(\text{Viagra} \mid \text{spam})$: *verosimilitud* (likelihood)
- $P(\text{Viagra})$: verosimilitud marginal

Ejemplo: spam y Viagra

Con una tabla de frecuencias de 100 correos:

Frequency	Viagra		Total
	Yes	No	
spam	4	16	20
ham	1	79	80
Total	5	95	100

Likelihood	Viagra		Total
	Yes	No	
spam	4 / 20	16 / 20	20
ham	1 / 80	79 / 80	80
Total	5 / 100	95 / 100	100

- $P(\text{spam}) = 20/100 = 0.2$; $P(\text{Viagra} \mid \text{spam}) = 4/20 = 0.2$;
 $P(\text{Viagra}) = 5/100 = 0.05$
- Entonces $P(\text{spam} \mid \text{Viagra}) = \frac{0.2 \cdot 0.2}{0.05} = 0.8$

¿Por qué “naive” (ingenuo)?

- Un filtro real usa muchas palabras, no una sola
- Naive Bayes hace dos supuestos “ingenuos” sobre las características:
 1. Que todas son **igual de importantes**
 2. Que son **independientes** entre sí
- Sabemos que rara vez es cierto (“Viagra”, “droga” y “prescripción” suelen ir juntas)
- Pero aun así el método **funciona muy bien** y suele ser el primer candidato

Varias características a la vez

- El supuesto de independencia simplifica enormemente el cálculo
- Con cuatro palabras $W_1, \dots, W_4$, la verosimilitud conjunta se factoriza:

$$P(\text{spam} \mid W_1 \cap \neg W_2 \cap \neg W_3 \cap W_4) \propto \\ P(W_1 \mid \text{spam}) \cdot P(\neg W_2 \mid \text{spam}) \cdot P(\neg W_3 \mid \text{spam}) \cdot P(W_4 \mid \text{spam}) \cdot P(\text{spam})$$

- Cada término sale de la tabla de frecuencias: un producto de proporciones simples

El estimador de Laplace

- ¿Qué pasa si una palabra aparece **0 veces** en una clase?
- Todo el producto se vuelve 0, y la probabilidad posterior también
- Eso no tiene sentido si las demás palabras sí apuntan a spam
- **Solución (Laplace):** sumar 1 a cada conteo de la tabla de frecuencias
- Así evitamos los ceros y el método se vuelve más robusto

Características numéricas

- Hasta aquí las características eran dicotómicas (la palabra está o no está)
- ¿Y si una característica es numérica? Por ejemplo, la hora de envío del correo
- Estrategia: **discretizar** la variable en categorías razonables
- Por ejemplo, dividir el día en franjas (madrugada, mañana, tarde, noche)
- **Al taller:** predecir el partido de legisladores de EE.UU. con la base `votos84.csv`

Resumen de la sesión

- El **machine learning** transforma datos en acción inteligente; aquí nos interesa **predecir**
- **KNN**: clasifica según los vecinos más cercanos. Simple, no paramétrico, sensible a la escala y a k
- **Naive Bayes**: clasifica de forma probabilística con el teorema de Bayes y un supuesto de independencia
- Ambos son **clasificadores**: predicen una categoría
- En la próxima sesión: SVM, árboles de decisión y random forests